First Year MCA

DBMS and NoSQL – Software Lab II (25P1513)

Assignments 1 to 19 – Step-by-Step Queries

Assignments 1–16: MySQL 8.x | Assignments 17–19: MongoDB (mongosh)

How to use this document

• **Step 0 – Setup** in each assignment creates the table(s) and inserts the sample records. Run it first.

• **Steps 1, 2, 3 …** match the numbered questions in your assignment sheet. Run them one at a time and note the output.

• SQL assignments run in order in MySQL Workbench or the mysql client, because later assignments reuse tables from earlier ones (Employee, Department, Student). Do the one-time database setup below first.

• Procedures, functions and triggers need `DELIMITER //` to be typed in the mysql client (not needed in Workbench's Navigator, but harmless in a script).

One-time database setup (MySQL)

```
DROP DATABASE IF EXISTS mca_lab;
CREATE DATABASE mca_lab;
USE mca_lab;
SET SQL_SAFE_UPDATES = 0;
```

One-time setup (MongoDB)

```
use("mca_lab");
```

Assignment 1: Student table (DDL / DML basics)

Step 0 – Setup (create table / insert records)

```
CREATE TABLE Student (
    RollNo INT PRIMARY KEY,
    Name   VARCHAR(50),
    Class  VARCHAR(20),
    Marks  INT
);

INSERT INTO Student VALUES
(1, 'Amit',  'FYMCA', 82),
(2, 'Sneha', 'FYMCA', 91),
(3, 'Rahul', 'FYMCA', 68),
(4, 'Nayan', 'FYMCA', 75),
(5, 'Pooja', 'FYMCA', 88);
```

Step 1. Display all records

```
SELECT * FROM Student;
```

Step 2. Students who scored more than 75

```
SELECT * FROM Student WHERE Marks > 75;
```

Step 3. Delete a record using RollNo

```
DELETE FROM Student WHERE RollNo = 3;
SELECT * FROM Student;
```

Step 4. Update marks of a particular student

```
UPDATE Student SET Marks = 80 WHERE RollNo = 4;
SELECT * FROM Student;
```

Step 5. Add a column Address

```
ALTER TABLE Student ADD COLUMN Address VARCHAR(100);
DESC Student;
-- (re-insert Rahul so later assignments still have 5 students)
INSERT INTO Student VALUES (3, 'Rahul', 'FYMCA', 68, 'Pune');
UPDATE Student SET Address = 'Pune' WHERE Address IS NULL;
SELECT * FROM Student;
```

Assignment 2: Emp table

Step 0 – Setup (create table / insert records)

```
CREATE TABLE Emp (
    Emp_id      INT,
    EName       VARCHAR(50),
    Department  VARCHAR(30),
    Designation VARCHAR(30),
    Jdate       DATE,
    Salary      DECIMAL(10,2)
);

INSERT INTO Emp VALUES
(1, 'Amit',  'IT',    'Programmer', '2025-06-01', 55000),
(2, 'Sneha', 'HR',    'Manager',    '2024-03-15', 62000),
(3, 'Rahul', 'Sales', 'Executive',  '2025-06-01', 30000),
(4, 'Nayan', 'IT',    'Programmer', '2023-09-10', 48000),
(5, 'Pooja', 'IT',    'Tester',     '2022-01-20', 52000);
```

Step 1. All records

```
SELECT * FROM Emp;
```

Step 2. Salary more than 50000

```
SELECT * FROM Emp WHERE Salary > 50000;
```

Step 3. Designation is Programmer

```
SELECT * FROM Emp WHERE Designation = 'Programmer';
```

Step 4. Joined on 2025-06-01

```
SELECT * FROM Emp WHERE Jdate = '2025-06-01';
```

Assignment 3: Primary key - Employee

Step 0 – Setup (create table / insert records)

```
CREATE TABLE Employee (
    Emp_id      INT PRIMARY KEY,
    EName       VARCHAR(50),
    DOB         DATE,
    Department  VARCHAR(30),
    Designation VARCHAR(30),
    Jdate       DATE,
    Salary      DECIMAL(10,2),
    Address     VARCHAR(50)
);

INSERT INTO Employee VALUES
(101, 'Nayan', '1995-03-12', 'Sales', 'Manager',    '2023-05-10', 45000, 'Pune'),
(102, 'Amit',  '1998-07-21', 'Sales', 'Executive',  '2024-02-15', 22000, 'Mumbai'),
(103, 'Sneha', '1996-11-05', 'IT',    'Programmer', '2022-08-01', 55000, 'Delhi'),
(104, 'Rahul', '1999-01-30', 'HR',    'Clerk',      '2025-06-01', 18000, 'Pune'),
(105, 'Pooja', '1997-09-17', 'Sales', 'Executive',  '2023-11-20', 28000, 'Nashik'),
(106, 'Kiran', '1994-04-25', 'IT',    'Programmer', '2021-12-12', 60000, 'Delhi'),
(107, 'Neha',  '2000-02-14', 'HR',    'Executive',  '2024-07-01', 26000, 'Mumbai'),
(108, 'Vikas', '1996-06-06', 'IT',    'Tester',     '2023-03-03', 32000, 'Pune');
```

Step 1. Name and address of employees in Sales

```
SELECT EName, Address FROM Employee WHERE Department = 'Sales';
```

Step 2. How many employees earn more than 25000

```
SELECT COUNT(*) AS Emp_Count FROM Employee WHERE Salary > 25000;
```

Step 3. Min, max, average, sum of salary

```
SELECT MIN(Salary) AS MinSal, MAX(Salary) AS MaxSal,
       AVG(Salary) AS AvgSal, SUM(Salary) AS TotalSal
FROM Employee;
```

Step 4. Salary between 20000 and 30000

```
SELECT * FROM Employee WHERE Salary BETWEEN 20000 AND 30000;
```

Step 5. 20% bonus to employees who joined before 1-1-24

```
UPDATE Employee SET Salary = Salary + (Salary * 0.20) WHERE Jdate < '2024-01-01';
SELECT * FROM Employee;
```

Step 6. Records as per Jdate

```
SELECT * FROM Employee ORDER BY Jdate;
```

Assignment 4: Foreign key - Department

Step 0 – Setup (create table / insert records)

```
CREATE TABLE Department (
    Dept_id INT PRIMARY KEY,
    DName   VARCHAR(40),
    Emp_id  INT,
    City    VARCHAR(30),
    FOREIGN KEY (Emp_id) REFERENCES Employee(Emp_id)
);

INSERT INTO Department VALUES
(1, 'Sales',       101, 'Pune'),
(2, 'Testing',     108, 'Pune'),
(3, 'Designing',   103, 'Delhi'),
(4, 'Maintenance', 106, 'Delhi'),
(5, 'Sales',       102, 'Mumbai'),
(6, 'Testing',     104, 'Delhi'),
(7, 'Designing',   NULL, 'Chennai');
```

Step 1. All records

```
SELECT * FROM Department;
```

Step 2. Departments located in Delhi

```
SELECT COUNT(*) AS Delhi_Departments FROM Department WHERE City = 'Delhi';
```

Step 3. Testing, Designing and Maintenance departments

```
SELECT * FROM Department WHERE DName IN ('Testing', 'Designing', 'Maintenance');
```

Step 4. Employees working in the same department and same city

```
SELECT d.DName, d.City, COUNT(e.Emp_id) AS Employees
FROM Department d
JOIN Employee e ON e.Emp_id = d.Emp_id
GROUP BY d.DName, d.City
HAVING COUNT(e.Emp_id) > 1;
-- (if no group has more than one, drop the HAVING line to see every group)
```

Assignment 5: Books

Step 0 – Setup (create table / insert records)

```
CREATE TABLE Books (
    BookID        INT PRIMARY KEY,
    Title         VARCHAR(100),
    Author        VARCHAR(60),
    Publisher     VARCHAR(50),
    ISBN          VARCHAR(20),
    YearPublished INT,
    Total_copies  INT,
    Price         DECIMAL(8,2)
);

INSERT INTO Books VALUES
(1, 'Algorithms Unlocked',        'Thomas Cormen',         'MIT Press',     '9780262518802', 2013,
    10, 450.00),
(2, 'Computer Networks',          'Andrew Tanenbaum',      'Pearson',       '9780132126953', 2010,
    8, 650.00),
(3, 'Modern Operating Systems',   'Andrew Tanenbaum',      'Pearson',       '9780133591620', 2014,
    6, 700.00),
(4, 'The C Programming Language', 'Brian Kernighan',       'Prentice Hall', '9780131103627', 1988,
    12, 500.00),
(5, 'Advanced Java',              'Herbert Schildt',       'McGraw Hill',   '9780071808552', 2012,
    7, 550.00),
(6, 'Database Management Systems','Raghu Ramakrishnan',    'McGraw Hill',   '9780072465631', 2002,
    9, 600.00);
```

Step 1. Titles starting with 'a'

```
SELECT * FROM Books WHERE Title LIKE 'a%';
```

Step 2. Titles containing 'er'

```
SELECT * FROM Books WHERE Title LIKE '%er%';
```

Step 3. Author ends with 'n'

```
SELECT * FROM Books WHERE Author LIKE '%n';
```

Step 4. Title whose second letter is 'o'

```
SELECT * FROM Books WHERE Title LIKE '_o%';
```

Step 5. Order of Title

```
SELECT * FROM Books ORDER BY Title;
```

Assignment 6: SET operations

Step 0 – Setup (create table / insert records)

```
-- (separate tables without FK so that every result is demonstrated)
CREATE TABLE EmpSet  (Emp_id INT, EName VARCHAR(50));
CREATE TABLE DeptSet (Emp_id INT, EName VARCHAR(50));

INSERT INTO EmpSet  VALUES (1,'Amit'),(2,'Sneha'),(3,'Rahul'),(4,'Nayan');
INSERT INTO DeptSet VALUES (3,'Rahul'),(4,'Nayan'),(5,'Pooja'),(6,'Kiran');
```

Step 1. All employees including duplicates

```
SELECT Emp_id, EName FROM EmpSet
UNION ALL
SELECT Emp_id, EName FROM DeptSet;
```

Step 2. All employees without duplicates

```
SELECT Emp_id, EName FROM EmpSet
UNION
SELECT Emp_id, EName FROM DeptSet;
```

Step 3. Common employees

```
SELECT Emp_id, EName FROM EmpSet
INTERSECT
SELECT Emp_id, EName FROM DeptSet;
-- Alternative for MySQL < 8.0.31:
-- SELECT e.Emp_id, e.EName FROM EmpSet e WHERE e.Emp_id IN (SELECT Emp_id FROM DeptSet);
```

Step 4. In EmpSet but not in DeptSet

```
SELECT Emp_id, EName FROM EmpSet
EXCEPT
SELECT Emp_id, EName FROM DeptSet;
-- Alternative: SELECT * FROM EmpSet WHERE Emp_id NOT IN (SELECT Emp_id FROM DeptSet);
```

Step 5. In DeptSet but not in EmpSet

```
SELECT Emp_id, EName FROM DeptSet
EXCEPT
SELECT Emp_id, EName FROM EmpSet;
-- Alternative: SELECT * FROM DeptSet WHERE Emp_id NOT IN (SELECT Emp_id FROM EmpSet);
```

Assignment 7: Count / GROUP BY (uses Employee from Assignment 3)

Step 1. Employees in each department

```
SELECT Department, COUNT(*) AS Total_Employees
FROM Employee GROUP BY Department;
```

Step 2. Departments having more than 2 employees

```
SELECT Department, COUNT(*) AS Total_Employees
FROM Employee GROUP BY Department HAVING COUNT(*) > 2;
```

Step 3. Total salary of each department

```
SELECT Department, SUM(Salary) AS Total_Salary
FROM Employee GROUP BY Department;
```

Assignment 8: String functions (uses Student from Assignment 1)

Step 1. Uppercase and lowercase

```
SELECT Name, UPPER(Name) AS Upper_Name, LOWER(Name) AS Lower_Name FROM Student;
```

Step 2. First 3 characters

```
SELECT Name, LEFT(Name, 3) AS First3 FROM Student;      -- or SUBSTRING(Name,1,3)
```

Step 3. Position of letter 'a' (0 = not present)

```
SELECT Name, INSTR(Name, 'a') AS Pos_of_a FROM Student; -- or LOCATE('a', Name)
```

Step 4. Length of each name

```
SELECT Name, LENGTH(Name) AS Name_Length FROM Student;  -- or CHAR_LENGTH(Name)
```

Assignment 9: User defined functions

Step 1. Create Employe and Student tables

```
DROP TABLE IF EXISTS Student;
CREATE TABLE Employe (
    Emp_id INT PRIMARY KEY,
    EName  VARCHAR(50),
    Salary DECIMAL(10,2)
);
CREATE TABLE Student (
    RollNo INT PRIMARY KEY,
    Name   VARCHAR(50),
    Marks  INT
);
INSERT INTO Employe VALUES
    (1,'Amit',30000),(2,'Sneha',45000),(3,'Rahul',25000),(4,'Nayan',52000),(5,'Pooja',38000);
INSERT INTO Student VALUES
    (1,'Amit',82),(2,'Sneha',91),(3,'Rahul',68),(4,'Nayan',75),(5,'Pooja',88);

DELIMITER //
```

Step 2. Square of a number

```
CREATE FUNCTION fn_square(n INT) RETURNS INT
DETERMINISTIC
BEGIN
    RETURN n * n;
END //
```

Step 3. Employee salary

```
CREATE FUNCTION fn_emp_salary(eid INT) RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE sal DECIMAL(10,2);
    SELECT Salary INTO sal FROM Employe WHERE Emp_id = eid;
    RETURN sal;
END //
```

Step 4. Marks of a student by roll number

```
CREATE FUNCTION fn_student_marks(roll INT) RETURNS INT
READS SQL DATA
BEGIN
    DECLARE m INT;
    SELECT Marks INTO m FROM Student WHERE RollNo = roll;
    RETURN m;
END //

DELIMITER ;

SELECT fn_square(7)          AS Square_of_7;
SELECT fn_emp_salary(2)      AS Salary_of_Emp2;
SELECT EName, fn_emp_salary(Emp_id) AS Salary FROM Employe;
SELECT fn_student_marks(4)   AS Marks_of_Roll4;
```

Assignment 10: Subqueries (uses Employee from Assignment 3)

Step 1. Salary greater than the average salary

```
SELECT * FROM Employee WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

Step 2. Employees with maximum and minimum salary

```
SELECT * FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee)
   OR Salary = (SELECT MIN(Salary) FROM Employee);
```

Step 3. Salary greater than Nayan's salary

```
SELECT * FROM Employee
WHERE Salary > (SELECT Salary FROM Employee WHERE EName = 'Nayan');
```

Assignment 11: Joins (Employee and Department)

Step 1. INNER JOIN - only matching records

```
SELECT e.Emp_id, e.EName, d.Dept_id, d.DName, d.City
FROM Employee e INNER JOIN Department d ON e.Emp_id = d.Emp_id;
```

Step 2. LEFT JOIN - all from left, matching from right

```
SELECT e.Emp_id, e.EName, d.Dept_id, d.DName, d.City
FROM Employee e LEFT JOIN Department d ON e.Emp_id = d.Emp_id;
```

Step 3. RIGHT JOIN - all from right, matching from left

```
SELECT e.Emp_id, e.EName, d.Dept_id, d.DName, d.City
FROM Employee e RIGHT JOIN Department d ON e.Emp_id = d.Emp_id;
```

Step 4. CROSS JOIN - all possible combinations

```
SELECT e.EName, d.DName
FROM Employee e CROSS JOIN Department d;
```

Assignment 12: Stored procedures

Step 0 – Setup (create table / insert records)

```
CREATE TABLE stud (
    stud_id INT PRIMARY KEY,
    Name    VARCHAR(50),
    Marks   INT,
    City    VARCHAR(30)
);
INSERT INTO stud VALUES
(1, 'Amit',  82, 'Pune'),
(2, 'Sneha', 91, 'Mumbai'),
(3, 'Rahul', 35, 'Pune'),
(4, 'Nayan', 55, 'Delhi'),
(5, 'Pooja', 66, 'Nashik');

DELIMITER //
```

Step 1. Display all students

```
CREATE PROCEDURE DisplayStudents()
BEGIN
    SELECT * FROM stud;
END //
```

Step 2. GetCity : employees of a particular city (Address column of Employee)

```
CREATE PROCEDURE GetCity(IN cityname VARCHAR(50))
BEGIN
    SELECT * FROM Employee WHERE Address = cityname;
END //
```

Step 3. Display numbers 1 to 10

```
CREATE PROCEDURE DisplayTen()
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE s VARCHAR(100) DEFAULT '';
    WHILE i <= 10 DO
        SET s = CONCAT(s, i, ' ');
        SET i = i + 1;
    END WHILE;
    SELECT TRIM(s) AS Numbers;
END //
```

Step 4. Grade using ELSEIF, by roll number

```
CREATE PROCEDURE CheckGrade(IN roll INT)
BEGIN
    DECLARE m INT;
    SELECT Marks INTO m FROM stud WHERE stud_id = roll;
    IF m IS NULL THEN
        SELECT 'Student not found' AS Result;
    ELSEIF m >= 75 THEN
        SELECT m AS Marks, 'Distinction' AS Grade;
    ELSEIF m >= 60 THEN
        SELECT m AS Marks, 'First Class' AS Grade;
    ELSEIF m >= 40 THEN
        SELECT m AS Marks, 'Pass' AS Grade;
    ELSE
        SELECT m AS Marks, 'Fail' AS Grade;
    END IF;
END //
```

Step 5. Colour using CASE

```
CREATE PROCEDURE ShowColor(IN num INT)
BEGIN
    CASE num
        WHEN 1 THEN SELECT 'Red'    AS Color;
        WHEN 2 THEN SELECT 'Green'  AS Color;
        WHEN 3 THEN SELECT 'Blue'   AS Color;
        WHEN 4 THEN SELECT 'Yellow' AS Color;
        ELSE        SELECT 'Invalid number' AS Color;
    END CASE;
END //

DELIMITER ;

CALL DisplayStudents();
CALL GetCity('Pune');
CALL DisplayTen();
CALL CheckGrade(3);
CALL ShowColor(2);
```

Assignment 13: Views

Step 0 – Setup (create table / insert records)

```
CREATE TABLE employes (
    Emp_id     INT PRIMARY KEY,
    EName      VARCHAR(50),
    Department VARCHAR(30),
    Salary     DECIMAL(10,2)
);
INSERT INTO employes VALUES
(1, 'Amit',  'IT',    30000),
(2, 'Sneha', 'HR',    45000),
(3, 'Rahul', 'Sales', 25000),
(4, 'Nayan', 'IT',    52000),
(5, 'Pooja', 'IT',    38000);
```

Step 1. Create a view and display records through it

```
CREATE VIEW emp_view AS
SELECT Emp_id, EName, Salary FROM employes WHERE Department = 'IT';
SELECT * FROM emp_view;
```

Step 2. Modify the view and see the effect on the original table

```
CREATE OR REPLACE VIEW emp_view AS
SELECT Emp_id, EName, Department, Salary FROM employes WHERE Salary > 30000;
SELECT * FROM emp_view;

UPDATE emp_view SET Salary = Salary + 5000 WHERE Emp_id = 4;   -- change through view
SELECT * FROM employes;                                         -- original table reflects it
```

Assignment 14: Index

Step 0 – Setup (create table / insert records)

```
DROP TABLE IF EXISTS student;
CREATE TABLE student (
    Roll_No INT,
    Name    VARCHAR(50),
    Marks   INT,
    Class   VARCHAR(20),
    Address VARCHAR(50)
);
INSERT INTO student VALUES
(1, 'Amit',  82, 'FYMCA', 'Pune'),
(2, 'Sneha', 91, 'FYMCA', 'Mumbai'),
(3, 'Rahul', 68, 'FYMCA', 'Pune'),
(4, 'Nayan', 75, 'FYMCA', 'Delhi'),
(5, 'Pooja', 88, 'FYMCA', 'Nashik');
```

Step 1. Create index

```
CREATE INDEX idx_student_name ON student(Name);
```

Step 2. Display index

```
SHOW INDEX FROM student;
```

Assignment 15: Triggers

Step 0 – Setup (create table / insert records)

```
DROP TABLE IF EXISTS student;
CREATE TABLE student (
    Roll_No INT PRIMARY KEY,
    Name    VARCHAR(50),
    Marks   INT,
    Class   VARCHAR(20),
    Address VARCHAR(50)
);
INSERT INTO student VALUES
(1, 'Amit',  82, 'FYMCA', 'Pune'),
(2, 'Sneha', 91, 'FYMCA', 'Mumbai'),
(3, 'Rahul', 68, 'FYMCA', 'Pune'),
(4, 'Nayan', 75, 'FYMCA', 'Delhi'),
(5, 'Pooja', 88, 'FYMCA', 'Nashik');

-- Backup table
CREATE TABLE studbackup (
    Roll_No INT,
    Name    VARCHAR(50),
    Marks   INT,
    Class   VARCHAR(20),
    Address VARCHAR(50)
);

-- Extra column used by the Pass/Fail trigger (Q4)
ALTER TABLE student ADD COLUMN Result VARCHAR(10);

DELIMITER //
```

Step 1. Fire on INSERT : copy new row to backup

```
CREATE TRIGGER trg_after_insert
AFTER INSERT ON student
FOR EACH ROW
BEGIN
    INSERT INTO studbackup VALUES (NEW.Roll_No, NEW.Name, NEW.Marks, NEW.Class, NEW.Address);
END //
```

Step 2. Fire on DELETE : keep the deleted row in backup

```
CREATE TRIGGER trg_after_delete
AFTER DELETE ON student
FOR EACH ROW
BEGIN
    INSERT INTO studbackup VALUES (OLD.Roll_No, OLD.Name, OLD.Marks, OLD.Class, OLD.Address);
END //
```

Step 3. Fire on UPDATE : keep the old values in backup

```
CREATE TRIGGER trg_after_update
AFTER UPDATE ON student
FOR EACH ROW
BEGIN
    INSERT INTO studbackup VALUES (OLD.Roll_No, OLD.Name, OLD.Marks, OLD.Class, OLD.Address);
END //
```

Step 4. Pass / Fail check on marks

```
CREATE TRIGGER trg_result
BEFORE INSERT ON student
FOR EACH ROW
BEGIN
    IF NEW.Marks >= 40 THEN
        SET NEW.Result = 'Pass';
    ELSE
        SET NEW.Result = 'Fail';
    END IF;
END //
```

Step 5. Check salary of employee (table employes from Assignment 13):

```
--    salary below 10000 is rejected with an error
CREATE TRIGGER trg_check_salary
BEFORE INSERT ON employes
FOR EACH ROW
BEGIN
    IF NEW.Salary < 10000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be at least 10000';
    END IF;
END //
```

Step 6. Name is stored in uppercase on insert

```
CREATE TRIGGER trg_upper_name
BEFORE INSERT ON student
FOR EACH ROW
BEGIN
    SET NEW.Name = UPPER(NEW.Name);
END //

DELIMITER ;

-- Test the triggers
INSERT INTO student (Roll_No, Name, Marks, Class, Address) VALUES (6, 'kiran', 35, 'FYMCA',
    'Pune');
UPDATE student SET Marks = 60 WHERE Roll_No = 2;
DELETE FROM student WHERE Roll_No = 5;
SELECT * FROM student;
SELECT * FROM studbackup;

INSERT INTO employes VALUES (6, 'Test', 'IT', 20000);    -- accepted
-- INSERT INTO employes VALUES (7, 'Low', 'IT', 5000);   -- fails: Salary must be at least 10000
```

Assignment 16: Cursors (uses Employee from Assignment 3)

Step 0 – Setup (create table / insert records)

```
DELIMITER //
```

Step 1. Cursor to display total salary

```
CREATE PROCEDURE cur_total_salary()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE sal  DECIMAL(10,2);
    DECLARE total DECIMAL(12,2) DEFAULT 0;
    DECLARE c CURSOR FOR SELECT Salary FROM Employee;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN c;
    read_loop: LOOP
        FETCH c INTO sal;
        IF done = 1 THEN LEAVE read_loop; END IF;
        SET total = total + sal;
    END LOOP;
    CLOSE c;
    SELECT total AS Total_Salary;
END //
```

Step 2. Cursor to count total number of employees

```
CREATE PROCEDURE cur_count_employees()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE eid  INT;
    DECLARE cnt  INT DEFAULT 0;
    DECLARE c CURSOR FOR SELECT Emp_id FROM Employee;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN c;
    read_loop: LOOP
        FETCH c INTO eid;
        IF done = 1 THEN LEAVE read_loop; END IF;
        SET cnt = cnt + 1;
    END LOOP;
    CLOSE c;
    SELECT cnt AS Total_Employees;
END //
```

Step 3. Cursor to show employees with salary more than 30000

```
CREATE PROCEDURE cur_high_salary()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE nm   VARCHAR(50);
    DECLARE sal  DECIMAL(10,2);
    DECLARE c CURSOR FOR SELECT EName, Salary FROM Employee WHERE Salary > 30000;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    DROP TEMPORARY TABLE IF EXISTS tmp_high;
    CREATE TEMPORARY TABLE tmp_high (EName VARCHAR(50), Salary DECIMAL(10,2));

    OPEN c;
    read_loop: LOOP
        FETCH c INTO nm, sal;
        IF done = 1 THEN LEAVE read_loop; END IF;
        INSERT INTO tmp_high VALUES (nm, sal);
    END LOOP;
    CLOSE c;
    SELECT * FROM tmp_high;
END //
```

Step 4. Cursor to print names in uppercase

```
CREATE PROCEDURE cur_upper_names()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE nm   VARCHAR(50);
    DECLARE c CURSOR FOR SELECT EName FROM Employee;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    DROP TEMPORARY TABLE IF EXISTS tmp_names;
    CREATE TEMPORARY TABLE tmp_names (UpperName VARCHAR(50));

    OPEN c;
    read_loop: LOOP
        FETCH c INTO nm;
        IF done = 1 THEN LEAVE read_loop; END IF;
        INSERT INTO tmp_names VALUES (UPPER(nm));
    END LOOP;
    CLOSE c;
    SELECT * FROM tmp_names;
END //

DELIMITER ;

CALL cur_total_salary();
CALL cur_count_employees();
CALL cur_high_salary();
CALL cur_upper_names();
```

NoSQL – MongoDB

Run in **mongosh**. Collections are created automatically on first insert.

Assignment 17: Book collection (Name, Addition [Edition], Author, Price)

Step 0 – Setup (create table / insert records)

```
db.Book.drop();

// Insert minimum 5 records (insertMany)
db.Book.insertMany([
  { Name: "C Program",       Addition: "3rd", Author: "Sudarshan",  Price: 450 },
  { Name: "Java Complete",   Addition: "2nd", Author: "Herbert",    Price: 600 },
  { Name: "Python Basics",   Addition: "1st", Author: "Guido",      Price: 350 },
  { Name: "DBMS Concepts",   Addition: "4th", Author: "Sudarshan",  Price: 700 },
  { Name: "Operating System",Addition: "5th", Author: "Galvin",     Price: 650 }
]);
```

Step 1. Insert one record

```
db.Book.insertOne({ Name: "Data Structures", Addition: "2nd", Author: "Tanenbaum", Price: 520 });
```

Step 2. Insert many records

```
db.Book.insertMany([
  { Name: "Computer Networks", Addition: "3rd", Author: "Forouzan", Price: 580 },
  { Name: "Software Engg",     Addition: "6th", Author: "Pressman", Price: 720 }
]);
```

Step 3. Find a particular record

```
db.Book.find({ Name: "Java Complete" });
db.Book.findOne({ Name: "Java Complete" });
```

Step 4. Books whose author is Sudarshan

```
db.Book.find({ Author: "Sudarshan" });
```

Step 5. Update author of book "C Program" to "balguru Swami"

```
db.Book.updateOne({ Name: "C Program" }, { $set: { Author: "balguru Swami" } });
db.Book.find({ Name: "C Program" });
```

Step 6. Delete a record

```
db.Book.deleteOne({ Name: "Software Engg" });
db.Book.find();   // verify
```

Assignment 18: CRUD operations on student and employe collections

Step 0 – Setup (create table / insert records)

```
db.student.drop();
db.employe.drop();
```

Step 1. Perform CRUD operations on student and employe collections

```
// ---- CREATE ----
db.student.insertMany([
  { RollNo: 1, Name: "Amit",  Class: "FYMCA", Marks: 82 },
  { RollNo: 2, Name: "Sneha", Class: "FYMCA", Marks: 91 },
  { RollNo: 3, Name: "Rahul", Class: "FYMCA", Marks: 68 },
  { RollNo: 4, Name: "Nayan", Class: "FYMCA", Marks: 75 },
  { RollNo: 5, Name: "Pooja", Class: "FYMCA", Marks: 88 }
]);

db.employe.insertMany([
  { Emp_id: 1, EName: "Amit",  Salary: 30000, Department: "IT",    City: "Pune"   },
  { Emp_id: 2, EName: "Sneha", Salary: 45000, Department: "HR",    City: "Mumbai" },
  { Emp_id: 3, EName: "Rahul", Salary: 25000, Department: "Sales", City: "Pune"   },
  { Emp_id: 4, EName: "Nayan", Salary: 52000, Department: "IT",    City: "Delhi"  },
  { Emp_id: 5, EName: "Pooja", Salary: 38000, Department: "IT",    City: "Nashik" }
]);

// ---- READ ----
db.student.find({ Marks: { $gt: 75 } });
db.employe.find({ Department: "IT" });

// ---- UPDATE ----
db.student.updateOne({ RollNo: 3 }, { $set: { Marks: 72 } });
db.employe.updateMany({ Department: "IT" }, { $inc: { Salary: 2000 } });

// ---- DELETE ----
db.student.deleteOne({ RollNo: 5 });
db.employe.deleteOne({ Emp_id: 3 });
```

Step 2. Display records of both collections

```
db.student.find().pretty();
db.employe.find().pretty();
```

Assignment 19: Operators on Employe collection

Step 0 – Setup (create table / insert records)

```
db.Employe.drop();
db.Employe.insertMany([
  { Emp_id: 1, EName: "ABC",   Salary: 30000, Department: "IT",    City: "Pune"   },
  { Emp_id: 2, EName: "PQR",   Salary: 45000, Department: "HR",    City: "Mumbai" },
  { Emp_id: 3, EName: "XYZ",   Salary: 55000, Department: "Sales", City: "Delhi"  },
  { Emp_id: 4, EName: "LMN",   Salary: 38000, Department: "IT",    City: "Nashik" },
  { Emp_id: 5, EName: "DEF",   Salary: 60000, Department: "HR",    City: "Pune"   }
]);
```

Step 1. Salary greater than 35000

```
db.Employe.find({ Salary: { $gt: 35000 } });
```

Step 2. Salary less than 50000

```
db.Employe.find({ Salary: { $lt: 50000 } });
```

Step 3. Records whose name is ABC and PQR (both names -> $in / $or)

```
db.Employe.find({ EName: { $in: ["ABC", "PQR"] } });
// same with $or:
db.Employe.find({ $or: [{ EName: "ABC" }, { EName: "PQR" }] });
```

Step 4. Records whose name is NOT ABC and PQR

```
db.Employe.find({ EName: { $nin: ["ABC", "PQR"] } });
// same with $nor:
db.Employe.find({ $nor: [{ EName: "ABC" }, { EName: "PQR" }] });
```